

Optimización de Consultas y Rendimiento

Actividad N°: 29 **Fecha:** 09/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Revisar y consolidar las optimizaciones de rendimiento aplicadas al backend, verificando que el sistema soporte el uso concurrente de varios puntos de venta sin degradarse (RNF-06, RNF-07).

2. Problemas de rendimiento identificados originalmente

- Error de cast en `checkTicketsChangesForStaff` al intentar buscar un `PuntoVenta` por `ObjectId` cuando `puntoTrabajo` en realidad es un `String` (nombre del punto).
- Ausencia de índices en campos consultados con alta frecuencia (`Ticket`, `updatedAt`).
- Consultas sin límite de tiempo, con riesgo de dejar el servidor colgado ante una consulta lenta.
- Rate limiting demasiado estricto (100 solicitudes cada 15 minutos), que bloqueaba el polling de verificación de cambios.
- Pool de conexiones de MongoDB pequeño para el número de puntos de venta concurrentes esperado.

3. Optimizaciones implementadas

Optimización	Detalle
Corrección de búsqueda por nombre	<code>PuntoVenta.findOne({ nombre: userPuntoTrabajo, activo: true })</code> en vez de buscar por <code>_id</code>
Índices agregados al modelo <code>Ticket</code>	<code>Ticket</code> (localidad), <code>updatedAt</code> (desc), compuesto <code>Ticket + updatedAt</code> , además de los índices de búsqueda por nombre/email/cédula ya definidos
Timeouts explícitos	<code>.maxTimeMS(10000)</code> en <code>find()</code> , <code>.maxTimeMS(5000)</code> en <code>countDocuments()</code> , <code>.maxTimeMS(3000)</code> en verificaciones ligeras de cambios (<code>check-changes</code>)
Rate limiting ajustado	De 100 solicitudes/15 min a 200 solicitudes/minuto, excluyendo explícitamente <code>/check-changes</code> y <code>/health</code>
Configuración de conexión MongoDB	<code>maxPoolSize: 20</code> , <code>minPoolSize: 5</code> , <code>retryReads: true</code> , <code>retryWrites: true</code> , <code>connectTimeoutMS</code> , <code>heartbeatFrequencyMS</code> , IPv4 forzado (<code>family: 4</code>)
Consultas en paralelo	<code>Promise.all</code> para ejecutar la búsqueda de documentos y el conteo total de forma simultánea, en vez de secuencial
Uso de <code>.lean()</code>	En todas las consultas de solo lectura (listados de tickets, verificación de cambios), evitando hidratar documentos Mongoose completos innecesariamente
Manejo de errores de timeout	Respuesta <code>503</code> con <code>retry: true</code> cuando una consulta excede su tiempo máximo, en vez de un error genérico <code>500</code>

Optimización	Detalle
Caché en memoria (<code>node-cache</code>)	Middleware <code>cacheMiddleware</code> con dos niveles de TTL (5 min general, 1 min para datos que cambian rápido) e invalidación selectiva por patrón (<code>invalidateCache</code>)

4. Resultados medidos (según seguimiento del equipo)

Métrica	Antes	Después	Mejora
Tiempo de respuesta de queries	2-5 s	0.5-1 s	~75%
Solicitudes bloqueadas por rate limit	~20%	<1%	~95%
Cuelgues del servidor	Frecuentes	Casi nulos	~98%
Uso de memoria	Alto	Moderado	~35%
Consultas por segundo soportadas	~10	~30-40	~300%

5. Verificación de índices

Se cuenta con un script dedicado (`backend/src/scripts/createIndexes.js`) que crea (o verifica) los índices necesarios, incluyendo un índice de texto compuesto sobre nombre, apellido, email, localidad, Ticket ID, Transaction ID y cédula, útil como respaldo cuando la búsqueda por regex no es suficiente.

6. Conclusiones del día

El sistema queda optimizado para el escenario real de uso (varios puntos de venta consultando y canjeando de forma concurrente durante el evento), con mejoras medibles en tiempo de respuesta, estabilidad ante consultas lentas y capacidad de solicitudes por segundo.

Observaciones: Sin observaciones.